MODULE 04 · DOMAIN 2 - TERRAFORM FUNDAMENTALS

State: What It Is, Why It Exists, What It Costs

The binding between your configuration and the real world, and the four jobs it does.

DOMAIN 2 of 8 COURSE WEIGHT 12% MODULE 4 of 27

admin@sthithapragna.com

State's primary purpose is to store **bindings** between remote objects and the resource instances in your configuration.

Four reasons state exists

- **Mapping to the real world.** Which remote object corresponds to which resource instance.
- **Metadata.** Resource dependencies, and which provider configuration was last used.
- **Performance.** A cached copy of every resource's attributes, so plans need not query everything.
- **Syncing.** A shared, lockable record so a team does not apply over each other.
- HashiCorp's position: state is **required**. Removing it would move enormous complexity somewhere else, not remove it.

> ON THE EXAM
>
> These four are the documented list, and questions are often built directly from them. Learn them as a set.

Mapping: why not just use tags?

- Early Terraform prototypes had **no state file** and identified resources by cloud tags instead.
- **Not all resources support tags,** and not all providers support them at all.
- So Terraform keeps its own structure, recording each object's identity when it creates it.
- **Each remote object must bind to exactly one resource instance.** Terraform guarantees this when it creates the object.
- Import the same object twice and the mapping is **ambiguous**. Terraform may then act unexpectedly against it.

> ON THE EXAM
>
> The one-to-one rule is why importing carefully matters. Creation is safe by construction; import is your responsibility.

Metadata: the part that surprises people

- State records **resource dependencies**, not just identities.
- Normally Terraform derives dependency order from the configuration, which is where the graph comes from.
- **But delete a resource block and its dependencies vanish from the configuration too.**
- Terraform still has to destroy that object, and still has to destroy things **in the right order**.
- So state retains the **most recent set of dependencies**. It also records which provider configuration was last used.

> ON THE EXAM
>
> Dependency metadata exists for deletion. Once the configuration is gone, state is the only place that ordering survives.

Performance: the attribute cache

- State caches the **attribute values** of every resource, not only its identity.
- HashiCorp calls this the **most optional** part of state. It is a performance feature, nothing more.
- **By default, every plan and apply refreshes every resource** by querying the provider.
- On a large estate that becomes slow, because it is one or more API calls per resource, every time.
- **-refresh=false** skips the query and plans from the cache. Faster, and the plan may be based on stale data.

> ON THE EXAM
>
> Refreshing is the default because correctness beats speed. Turning it off is a deliberate trade, not an optimisation.

Three things called refresh

WHAT YOU WRITE	WHAT IT DOES
`Default plan or apply`	Refreshes every resource, then compares against configuration
`-refresh=false`	Skips the query. Plans from cached attributes, which may be stale
`plan -refresh-only`	Compares state against reality only. Proposes no infrastructure changes
`apply -refresh-only`	Writes those findings into state, after you approve them
`terraform refresh`	Deprecated since 0.15.4. Updates state with no preview and no prompt

ON THE EXAM

refresh=false skips reality. refresh-only looks at reality and nothing else. Opposite meanings, one shared word.

Syncing: state is a shared resource

- **State on a laptop is state one person has.** A second engineer plans against a different picture of reality.
- **Remote state** puts one authoritative copy where everyone and every pipeline reads it.
- **State locking** stops two applies running at once, which is how state gets corrupted.
- Locking is per backend. Not every backend supports it, and that is a real selection criterion.
- Backends, locking and migration are module sixteen. Here, only the reason they exist.

> ON THE EXAM
>
> Two people applying against separate local state files will each destroy what the other created. Sharing state is not optional for a team.

The three inputs to every plan

01

Configuration

What you have declared should exist. The desired state.

02

State

What Terraform believes exists, and which object is which.

03

Reality

What the provider reports right now, read during refresh.

04

Plan

The difference, expressed as creates, updates, replacements and destroys.

ON THE EXAM

A plan is a three-way comparison, not a two-way one. Most confusing plan output comes from forgetting the middle input.

Desired state or current state?

DESIRED STATE

Your configuration files

- Written by you, in HCL
- Version-controlled and reviewed
- Describes the end result
- Changing it changes nothing on its own

CURRENT STATE

The state file, refreshed

- Written by Terraform, never by hand
- Records identities and attributes
- Describes what exists now
- Changing it changes nothing either

THE TIEBREAKER

Neither file touches infrastructure. Only apply does. Editing state to 'fix' something changes what Terraform believes, not what is true.

What the file actually looks like

- **JSON**, in a documented but internal format. It is not a configuration file and not written in HCL.
- Carries a **version** for the format, a **serial** that increments on change, and a **lineage** identifying the state's history.
- By default it is **terraform.tfstate** in the working directory, with **terraform.tfstate.backup** holding the previous version.
- **The format is internal.** HashiCorp reserves the right to change it, so never write tooling that parses it directly.
- Read it with **terraform show** or the state commands, which give you a supported interface.

> ON THE EXAM
>
> That it is JSON does not make it a file you may edit. Serial and lineage exist so Terraform can detect a state that has been swapped or rolled back.

State holds your secrets in plain text

- **Any attribute a provider returns is stored, including passwords, keys and tokens.** In plain text.
- **Marking a variable sensitive does not change this.** It hides values in CLI output. State is unaffected.
- So HashiCorp's guidance is blunt: **treat state itself as sensitive data**.
- Practically: encrypt it at rest, restrict who can read the backend, and never commit it to a repository.
- **Ephemeral values and write-only arguments** keep specific secrets out of state entirely. Module thirteen.

> ON THE EXAM
>
> sensitive controls what is printed. It has never controlled what is stored. That gap is the single most common misconception about state.

When state is lost

- **Terraform stops knowing your infrastructure exists.** The resources keep running.
- The next plan proposes to **create everything from scratch**.
- **terraform.tfstate.backup** holds the previous version. A bad write, not a deleted backend.
- Real protection is a **versioned remote backend**.
- Last resort: **import every resource back**. Slow and manual. Module nineteen.

> ON THE EXAM
>
> Losing state does not destroy anything. It destroys Terraform's knowledge, which is worse, because the next apply looks reasonable.

What state is not

A TEMPTING BELIEF	THE REALITY
State is a backup of my infrastructure	It records identities and attributes, and recreates nothing
Marking values sensitive protects state	sensitive affects output only. State is plain text
State is optional for small projects	Terraform always uses state. Local state is still state
I can hand-edit state to fix a problem	You change what Terraform believes. Reality does not move
The lock file and the state file are related	Unrelated. Provider selections versus infrastructure

ON THE EXAM

These distractors are all true things about state, stretched slightly too far.

Reading the scenario

THE WORDING SAYS	IT IS POINTING AT
Plans take far too long on a large estate	The refresh step, and -refresh=false as a trade
A password appeared somewhere it should not	State stores attributes in plain text
Two engineers overwrote each other's work	Local state, and the need for remote state plus locking
A resource was destroyed in the wrong order	Dependency metadata, which state retains for deletion
The same object was imported twice	The one-to-one binding rule, now ambiguous
Why can Terraform not just query the cloud?	Tags are not universal, and deletion ordering would be lost

ON THE EXAM

Ask which of the four purposes the scenario is stressing. Mapping, metadata, performance or syncing.

A team removes an application server and the private subnet it sat in from their configuration in a single commit, deleting both resource blocks. The subnet cannot be deleted while the server still exists.

The apply succeeds. The server is destroyed first, then the subnet. A junior engineer asks how Terraform knew the ordering, given that neither resource block still exists to describe the relationship.

What made the correct ordering possible?

A Terraform read the dependency from the previous commit in version control

B Terraform retains the most recent set of resource dependencies in state

C The provider rejected the subnet deletion and Terraform retried in a different order

D Terraform destroys resources in reverse alphabetical order by resource address

◆ PAUSE AND DECIDE

What made the correct ordering possible?

A Read from version control

Terraform has no awareness of Git. It sees only the files in the working directory, and those no longer describe either resource.

B Dependencies retained in state

Correct, and this is precisely why state stores dependency metadata. Configuration cannot describe the ordering of something it no longer contains.

C Failed and retried

Terraform does not discover ordering by trial and error. A dependency violation surfaces as an error, not as an automatic reordering.

D Reverse alphabetical

Invents a rule, and a plausible-sounding one. Ordering always comes from the dependency graph, never from names.

THE SIGNAL

The graph normally comes from configuration. For deletion it cannot, which is exactly the gap dependency metadata fills.

admin@sthithapragna.com

A configuration manages about four thousand resources. A plan now takes over twenty minutes before printing anything, and engineers have started skipping plans during incidents.

A colleague suggests always running plan with -refresh=false, and says it makes plans nearly instant.

How should the team evaluate this suggestion?

A Adopt it as the default. Refresh is a performance feature and adds no correctness

B Understand it as a trade: faster plans, computed against cached attributes that may be stale

C Reject it. Terraform cannot produce a plan without refreshing first

D Adopt it, because state is refreshed during apply regardless of the plan flag

◆ PAUSE AND DECIDE

admin@sthithapragna.com

How should the team evaluate this suggestion?

A No correctness cost

Inverts what the cache is for. The cache is the performance feature; refreshing is what makes the plan reflect reality.

B A trade, understood

Correct. The flag is legitimate and the cost is real: any change made outside Terraform since the last refresh is invisible to that plan.

C Cannot plan without refresh

False. Terraform plans happily from cached attributes, which is exactly what the flag asks it to do.

D Apply refreshes anyway

Mixes two things. A saved plan is applied as written; the value of seeing an accurate plan is lost whatever apply does afterwards.

THE SIGNAL

Skipping plans during incidents is the planted detail. It says drift is likely, which is exactly when a stale plan is most dangerous.

admin@sthithapragna.com

> A configuration creates a managed database with a generated administrator password. The input variable holding it is declared with sensitive = true, and plan output shows the value as sensitive rather than printing it.
>
> During a security review, an auditor opens the state file in the remote backend and reads the password in plain text. The team is surprised, having believed the sensitive marking protected it.

What is the correct explanation?

A. The sensitive marking failed because it was applied to a variable rather than an output

B. sensitive affects displayed output only. State stores attribute values in plain text regardless

C. The backend was misconfigured; correctly configured backends encrypt sensitive attributes individually

D. The password was written to state because the plan was run with -refresh=false

◆ PAUSE AND DECIDE

What is the correct explanation?

A Wrong place for sensitive

Variables and outputs both accept sensitive, and it behaves the same way in both. The marking worked exactly as designed.

B Output only, never storage

Correct. sensitive suppresses display. Terraform stores whatever the provider returns, in plain text, which is why HashiCorp says to treat state as sensitive.

C Backend misconfiguration

No backend encrypts individual attributes. Encryption at rest protects the whole file, and anyone authorised to read it still sees plain text.

D Caused by -refresh=false

Unrelated. Refresh controls whether attributes are re-read; it has no bearing on whether they are stored.

THE SIGNAL

Nothing malfunctioned. The team's mental model was wrong, and the tool behaved exactly as documented.

admin@sthithapragna.com

> A newly hired architect proposes removing state files entirely. Her argument is that the cloud provider already knows what exists, so Terraform should query the provider on every run and compare the results with the configuration directly.
>
> She points out this would eliminate state locking, state corruption, and the risk of secrets sitting in a state file.

What is the strongest technical objection?

A Provider APIs are too slow to query, so this would be impractical at any scale

B Terraform could not identify which objects it manages, nor order deletions once configuration is removed

C The Terraform language has no way to express a comparison against live infrastructure

D Providers do not expose read operations, only create, update and delete

◆ PAUSE AND DECIDE

What is the strongest technical objection?

A APIs too slow

A real cost and the weakest objection, since it is only about speed. It is also the problem the attribute cache already exists to reduce.

B Identity and deletion ordering

Correct. It names both failures HashiCorp documents: tags are not universally supported, and deletion ordering has no source once configuration is gone.

C Language cannot express it

False. Data sources read live infrastructure routinely. Reading is not the difficulty; knowing which readings are yours is.

D No read operations

False, and refresh depends on reads. Every provider implements them.

THE SIGNAL

Her three complaints are all genuine. Every one of them is a cost of state, not an argument that state is unnecessary.

admin@sthithapragna.com

Module 4 recap

- **State stores bindings** between remote objects and resource instances. That is its primary purpose.
- **Four documented reasons:** mapping, metadata, performance, syncing.
- **Dependency metadata exists for deletion,** when configuration can no longer describe the ordering.
- **Refresh is the default and it is the correctness step.** The cache is the optimisation.
- **State holds secrets in plain text.** sensitive controls printing, never storage.
- **Editing state changes belief, not reality.** Only apply changes infrastructure.

> ON THE EXAM
>
> A plan compares three things, not two: configuration, state, and what the provider reports.